ROS Action Communication

Prof.: Dr. Oybek Eraliev

TA: Jasurbek Mamurov

Lecture Agenda

ROS1 Action Communication — What We'll Cover Today

01	Recap: ROS1 Communication Basics
02	What is ROS1 actionlib?
03	Action Architecture & Components
04	Service vs Action: Key Differences
05	Custom .action File Syntax
06	Example 1: Navigation Action (rospy)
07	Example 2: Robot Arm Pick & Place
08	Action Goal Lifecycle & States
09	Feedback & Cancellation
10	Class Activity

ROS1 Communication Patterns — Quick Recap

Three core paradigms in ROS1

📡 Topics (Pub / Sub)

Async, one-to-many
No response
Good for sensor streams
rospy.Publisher / Subscriber

🔁 Services (Request / Response)

Synchronous, one-to-one
Blocks until response
Good for quick queries
rospy.ServiceProxy

🎯 Actions (actionlib)

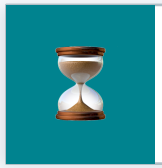
Async, non-blocking
Feedback + cancellable
Long-running tasks
actionlib.SimpleActionClient

⭐ TODAY'S FOCUS

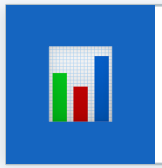
What is ROS1 actionlib?

The standard way to handle long-running, preemptable tasks

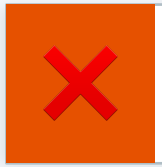
actionlib provides a standardised interface for preemptable tasks that:



Take a long time to complete (seconds to minutes)



Need to report progress with periodic feedback messages



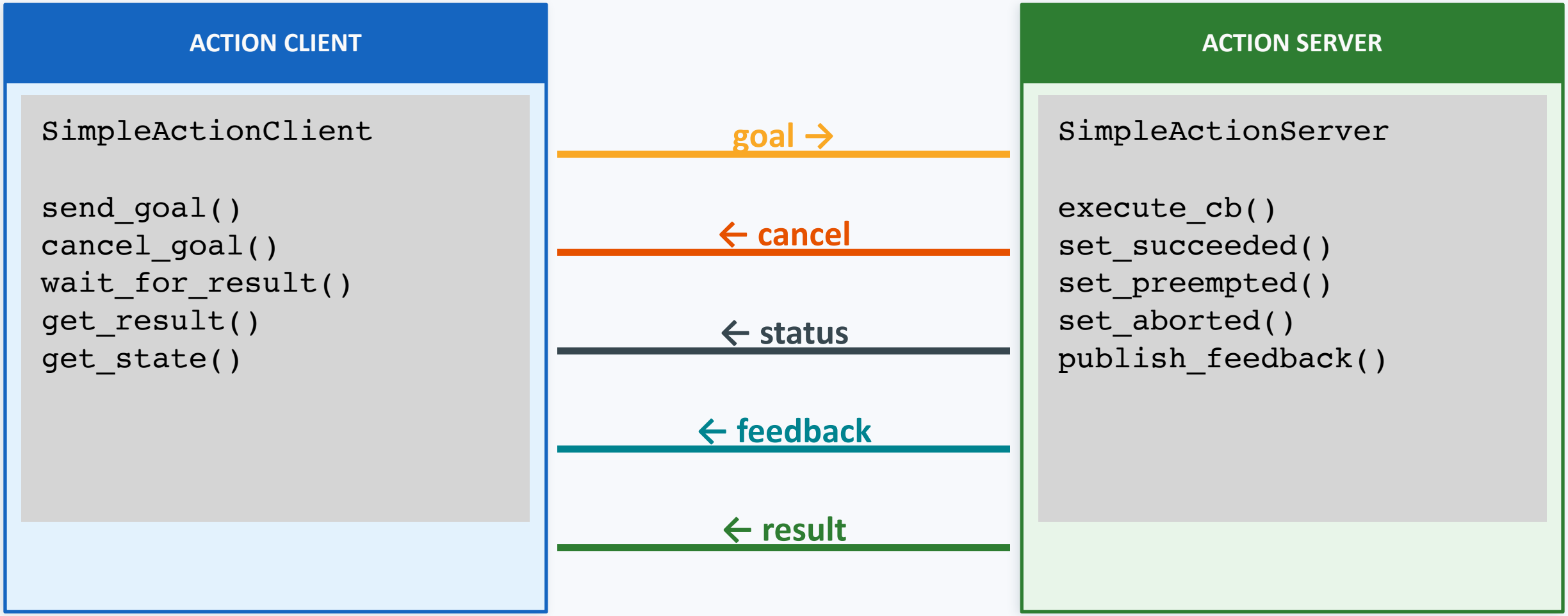
Must be preemptable — client can cancel at any time

Return a final result when the task finishes

 *Real-world analogy: Telling a robot "Navigate to the kitchen." It may take 30 s, you want status updates every second, and you can cancel if priorities change.*

actionlib Architecture — Client & Server

Five ROS topics created automatically under the action namespace



All 5 topics live under: /action_name/goal /cancel /status /feedback /result

Service vs Action Communication

Yes — rospy fully supports both; actionlib replaces rclpy action API

✅ **YES — in ROS1, rospy replaces rclpy entirely. Actions use the actionlib library (not rclpy_action).**

Feature	rospy Service	actionlib Action
Library	rospy	actionlib
Pattern	Request / Response	Goal / Feedback / Result
Blocking?	Yes — blocks caller	No — asynchronous
Feedback	❌ None	✅ Continuous via callback
Cancellation	❌ Not supported	✅ cancel_goal()
State machine	❌ No	✅ PENDING → ACTIVE → DONE
File extension	.srv	.action
Server class	rospy.Service	SimpleActionServer
Client class	rospy.ServiceProxy	SimpleActionClient

Service vs Action — Flow Comparison

Visual step-by-step execution flow

rospy SERVICE Pattern

1. Client sends Request
2. Server BLOCKS immediately
3. **Client BLOCKED — waiting**
4. Server processes task
5. Server sends Response
6. Client unblocks & continues

actionlib Action Pattern

1. Client sends Goal
2. Server accepts (non-blocking)
3. **Client continues other work ✅**
4. Server publishes Feedback...
5. Server publishes Feedback...
6. Server sends Final Result

Custom Action File (.action)

Three sections separated by --- delimiters

Every .action file is split into exactly three sections:

```
# ── GOAL ──────────────────────────────────────────────────────────
geometry_msgs/PoseStamped  target_pose
float32                    speed_limit
---
# ── RESULT ────────────────────────────────────────────────────────
bool     success
string   message
float32  total_distance_traveled
---
# ── FEEDBACK ──────────────────────────────────────────────────────
geometry_msgs/Pose  current_pose
float32             distance_remaining
float32             estimated_time_remaining
```

Example 1: Navigate.action File

robot_actions/action/Navigate.action — move a robot to a target

File path: catkin_ws/src/robot_actions/action/Navigate.action

```
# Navigate.action
# ── GOAL ──────────────────────────────────────────────────────────
geometry_msgs/PoseStamped  target_pose      # Destination (frame_id + pose)
float32                    max_speed        # Maximum speed in m/s
bool                       avoid_obstacles  # Enable obstacle avoidance
---
# ── RESULT ────────────────────────────────────────────────────────
bool     success           # True if robot reached goal
string   status_message    # Human-readable status
float32  total_distance    # Path length actually travelled (m)
float32  time_elapsed      # Wall-clock time taken (s)
---
# ── FEEDBACK ──────────────────────────────────────────────────────
geometry_msgs/Point  current_position     # Robot's current XY position
float32              distance_to_goal     # Remaining distance (m)
float32              progress_percentage  # 0.0 – 100.0
string               current_status       # e.g. "Navigating", "Rotating"
```

💡 *Use case: Delivery robot heading from charging dock to Room 302 — operator watches progress, cancels if priority changes.*

Example 1: Navigation Action Server (rospy)

Using actionlib.SimpleActionServer

```
#!/usr/bin/env python
import rospy
import actionlib
import time
from robot_actions.msg import NavigateAction, NavigateGoal, NavigateFeedback, NavigateResult

class NavigationServer:
    def __init__(self):
        # Create the SimpleActionServer
        self.server = actionlib.SimpleActionServer(
            'navigate',            # action name (namespace)
            NavigateAction,        # action type
            execute_cb=self.execute_cb,
            auto_start=False)
        self.server.start()
        rospy.loginfo("NavigationServer ready.")

    def execute_cb(self, goal):
        rospy.loginfo(f"Goal received: navigate to {goal.target_pose.pose}")
        feedback = NavigateFeedback()
        result   = NavigateResult()

        for i in range(10):           # Simulate 10 steps toward goal
            # ── Check for preempt (cancel) request ──────────────────
            if self.server.is_preempt_requested():
                rospy.logwarn("Preempt requested — stopping.")
                result.success = False
                result.status_message = "Cancelled by client"
                self.server.set_preempted(result)
                return
```

Example 1: Navigation Action Server (rospy)

Using actionlib.SimpleActionServer

```
            # ── Publish feedback ──────────────────────────────────
            feedback.progress_percentage = float(i * 10)
            feedback.distance_to_goal    = 10.0 - float(i)
            feedback.current_status      = "Navigating"
            self.server.publish_feedback(feedback)
            rospy.sleep(0.5)

        result.success        = True
        result.total_distance = 15.3
        result.status_message = "Goal reached"
        self.server.set_succeeded(result)

if __name__ == '__main__':
    rospy.init_node('navigation_server')
    NavigationServer()
    rospy.spin()
```

Example 1: Navigation Action Client (rospy)

Sending a goal and receiving feedback

```
#!/usr/bin/env python
import rospy
import actionlib
from geometry_msgs.msg import PoseStamped
from robot_actions.msg import NavigateAction, NavigateGoal

def feedback_cb(feedback):
    rospy.loginfo(f"Progress: {feedback.progress_percentage:.1f}%  "
                  f"Distance: {feedback.distance_to_goal:.2f} m  "
                  f"Status: {feedback.current_status}")

def main():
    rospy.init_node('navigation_client')

    # Create client and wait for server
    client = actionlib.SimpleActionClient('navigate', NavigateAction)
    rospy.loginfo("Waiting for action server...")
    client.wait_for_server()

    # Build goal
    goal = NavigateGoal()
    goal.target_pose.header.frame_id = "map"
    goal.target_pose.pose.position.x = 5.0
    goal.target_pose.pose.position.y = 3.0
    goal.max_speed       = 0.5
    goal.avoid_obstacles = True
```

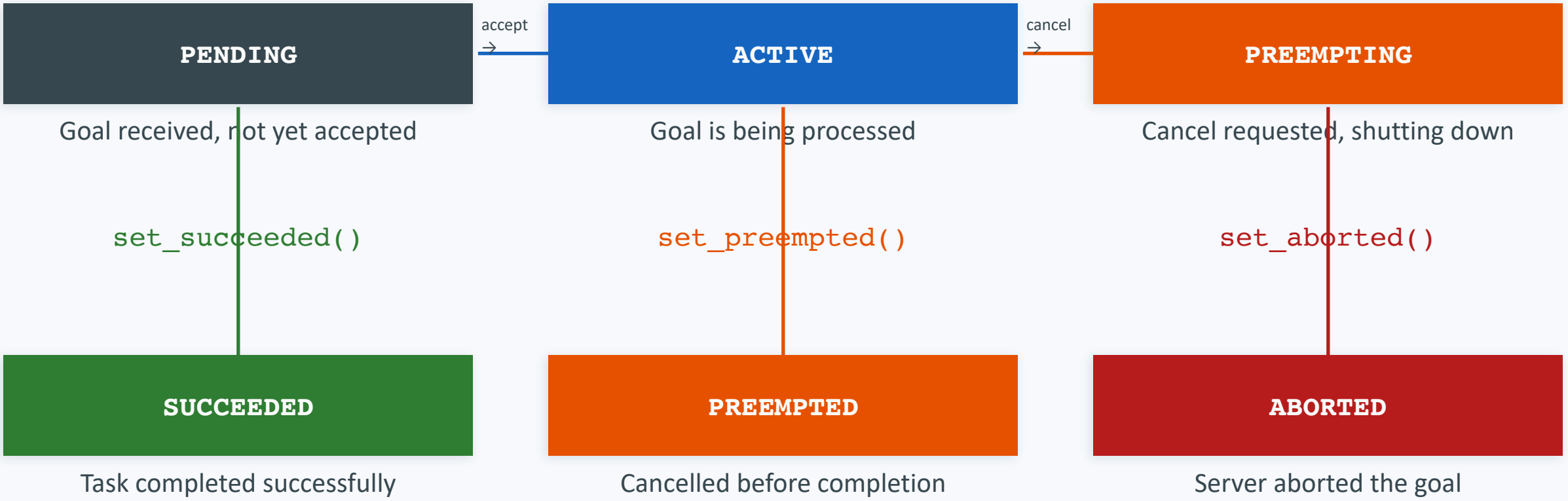
Example 1: Navigation Action Client (rospy)

Sending a goal and receiving feedback

```
    # Send goal with feedback callback
    client.send_goal(goal, feedback_cb=feedback_cb)
    rospy.loginfo("Goal sent — waiting for result...")

    # Wait (non-blocking — you could do other work here)
    client.wait_for_result(rospy.Duration(60.0))   # timeout: 60 s

    result = client.get_result()
    state  = client.get_state()    # actionlib.GoalStatus constants
    rospy.loginfo(f"State: {state}  Success: {result.success}  "
                  f"Distance: {result.total_distance:.1f} m")

if __name__ == '__main__':
    main()
```

Action Goal Lifecycle (GoalStatus)

State machine for every action goal in actionlib



📌 Terminal states (SUCCEEDED, PREEMPTED, ABORTED) are final — the goal cannot transition further. Always call exactly one terminal method.

Feedback & Preemption In Depth

Two mechanisms that make actionlib superior to services

FEEDBACK

- Published while the goal is ACTIVE
- Defined in 3rd section of .action file
- Server calls self.server.publish_feedback(fb)
- Client registers feedback_cb in send_goal()
- Informational only — does not pause execution
- Recommended frequency: 1–5 Hz

PREEMPTION (Cancel)

- Client calls client.cancel_goal()
- Server polls is_preempt_requested()
- Server calls set_preempted(result)
- Server is responsible for safe stop!
- Critical for physical hardware safety
- Always implement in hardware-facing nodes

⚠ **Never ignore is_preempt_requested() in a loop — failing to check may cause the robot to continue an unsafe action.**

Setting Up a Custom Action Package (catkin)

Package.xml, CMakeLists.txt and directory layout

package.xml — key dependencies:

```
<buildtool_depend>catkin</buildtool_depend>
<build_depend>actionlib</build_depend>
<build_depend>actionlib_msgs</build_depend>
<build_depend>geometry_msgs</build_depend>
<build_depend>message_generation</build_depend>
<exec_depend>actionlib</exec_depend>
<exec_depend>actionlib_msgs</exec_depend>
<exec_depend>rospy</exec_depend>
<exec_depend>message_runtime</exec_depend>
```

CMakeLists.txt — build rules:

```
find_package(catkin REQUIRED COMPONENTS
  actionlib actionlib_msgs
  geometry_msgs message_generation)

add_action_files(
  FILES
  Navigate.action
  PickPlace.action)

generate_messages(
  DEPENDENCIES
  actionlib_msgs geometry_msgs)

catkin_package(
  CATKIN_DEPENDS message_runtime)
```

Directory structure:

```
robot_actions/
├── action/
│   ├── Navigate.action
│   └── PickPlace.action
├── scripts/
│   ├── navigation_server.py
│   ├── navigation_client.py
│   └── pick_place_server.py
```

actionlib (ROS1) vs rclpy_action (ROS2)

Key differences for students migrating or comparing

Aspect	ROS1 actionlib	ROS2 rclpy_action
Client class	`SimpleActionClient`	`ActionClient`
Server class	`SimpleActionServer`	`ActionServer`
Send goal	`client.send_goal(goal, cb)`	`client.send_goal_async(goal)`
Wait for result	`client.wait_for_result()`	`Future / await`
Cancel goal	`client.cancel_goal()`	`goal_handle.cancel_goal_async()`
Check preempt	`is_preempt_requested()`	`is_cancel_requested`
Feedback method	`server.publish_feedback(fb)`	`goal_handle.publish_feedback(fb)`
Set succeeded	`server.set_succeeded(res)`	`goal_handle.succeed()`
Set aborted	`server.set_aborted(res)`	`goal_handle.abort()`
Build system	`catkin + CMakeLists`	`colcon + CMakeLists / setup.py`

Common Pitfalls & Best Practices

What goes wrong — and how to fix it

Missing preempt check

✗ Server ignores is_preempt_requested()

✓ Poll in every loop iteration

auto_start=True by default

✗ Server starts before callbacks are set

✓ Always pass auto_start=False

No feedback published

✗ Long task with zero progress updates

✓ Publish every 0.5–1 s minimum

Multiple terminal calls

✗ Calling set_succeeded after set_aborted

✓ Return immediately after any terminal

No timeout in client

✗ wait_for_result() blocks forever

✓ Pass rospy.Duration(N) as timeout

Missing message_generation

✗ catkin_make fails with unknown type

✓ Add message_generation to find_package

Debugging Actions — rostopic & rostopic Tricks

Inspecting action topics from the terminal

```
$ rostopic list | grep navigate
```

List all 5 action topics

```
$ rostopic echo /navigate/feedback
```

Print live feedback messages

```
$ rostopic echo /navigate/result
```

Print result when goal finishes

```
$ rostopic echo /navigate/status
```

Watch goal state transitions

```
$ rostopic pub /navigate/goal robot_actions/NavigateActionGoal \
  "{ goal: { max_speed: 0.5 } }"
```

Manually publish a goal from CLI

```
$ rosrun actionlib axclient.py /navigate
```

GUI client — send goals interactively!

CLASS ACTIVITY

- **Key requirements for your server**
  - Simulate a temperature reading each second using:
  - `import random` then `temp = random.uniform(18.0, 42.0)`
  - Track the highest temperature seen so far (for the result)
  - Set feedback.status to 'ALERT' if temp > alert_threshold, else 'OK'
  - Publish feedback every second
  - After monitor_duration seconds, call set_succeeded() with the result
  - Handle cancellation: call set_preempted() if requested

  - Your client should:
    - Create a TempMonitorGoal with monitor_duration=10 and alert_threshold=35.0
    - Send the goal and register a feedback_cb that prints current_temp and status
    - After wait_for_result(), print whether threshold_exceeded is True or False
    - Print the max_temp_seen from the result
    - modify the client to cancel the goal if 3 consecutive ALERT readings occur

Expected output (will vary due to random temps):

```
[Feedback] Temp: 22.3 C  Status: OK
[Feedback] Temp: 38.1 C  Status: ALERT
[Feedback] Temp: 29.7 C  Status: OK
...
[Result] Threshold exceeded: True
Max temp seen: 38.1 C
```

Lecture Summary — Key Takeaways

ROS1 Action Communication with actionlib & rospy

actionlib

ROS1's standard library for long-running, preemptable tasks

GoalStatus

PENDING → ACTIVE → SUCCEEDED / PREEMPTED / ABORTED

Architecture

Client sends Goal → Server executes → Feedback loop → Result

Server class

SimpleActionServer(name, Type, execute_cb, auto_start=False)

.action File

3-section: Goal / Result / Feedback, separated by ---

Client class

SimpleActionClient(name, Type) + wait_for_server()

vs Services

Actions are async, have goal states, and support cancellation (preempt)

Best practice

Always check is_preempt_requested(); publish feedback regularly

Further Reading & Q&A

ROS1 actionlib resources

Recommended Resources

- 📘 ROS Wiki — actionlib Tutorials: wiki.ros.org/actionlib/Tutorials
- 📗 actionlib Python API: wiki.ros.org/actionlib_python
- 📙 actionlib Detailed Description: wiki.ros.org/actionlib/DetailedDescription
- 📕 GitHub examples: github.com/ros/common_tutorials/tree/noetic/actionlib_tutorials
- 📹 YouTube — The Construct: ROS1 Actions from Scratch (rospy)
- 📒 Book: "Programming Robots with ROS" (Quigley, Gerkey, Smart) — Chapter 9

Questions? 🙋


Q
&
A

Thank you!